Hadoop Prt 2

Programmazione

Apache Hadoop è un framework open-source progettato per gestire e analizzare grandi quantità di dati (Big Data) distribuendoli su più computer collegati in rete.

L'idea alla base è semplice: invece di usare un singolo computer molto potente, Hadoop divide i dati e le operazioni tra molti nodi di un cluster, permettendo di elaborare dataset enormi in modo scalabile e relativamente economico.
Hadoop nasce nel contesto della crescita esponenziale dei dati digitali (log web, dati social, sensori, transazioni ecc.).

Le architetture tradizionali basate su database centralizzati non riuscivano più a gestire volumi così grandi.

Hadoop introduce quindi un modello distribuito che permette di archiviare e processare dati su centinaia o migliaia di macchine, mantenendo affidabilità anche se alcuni nodi del sistema falliscono.

Un altro aspetto fondamentale di Hadoop è il concetto di fault tolerance: i dati non vengono salvati in una sola posizione ma replicati su più nodi del cluster.

Se una macchina si guasta, il sistema continua a funzionare utilizzando le copie presenti su altri nodi.

Hadoop è strutturato in moduli principali, ognuno con un insieme di classi e responsabilità ben definite, le due macro-componenti fondamentali sono HDFS (storage distribuito) e MapReduce/YARN (elaborazione distribuita e gestione delle risorse).

È il layer di storage distribuito, progettato per gestire grandi quantità di dati su cluster.

Classi principali:

- **FileSystem :**

Classe astratta che rappresenta il filesystem distribuito. Fornisce metodi come open(), create(), delete(), listStatus(). È il punto di accesso principale ai file.

- **Path :**

Rappresenta il percorso di un file o directory in HDFS. È usato in tutte le operazioni di lettura/scrittura.

- **FSDataInputStream / FSDataOutputStream :**

Stream per leggere e scrivere file su HDFS.

- **NameNode :**

Gestisce i metadati (nomi file, struttura directory, posizione dei blocchi). Non contiene i dati veri e propri.

- **DataNode :**

Gestisce i blocchi fisici dei dati. Si occupa di lettura/scrittura e replica.

- **Block :**

Unità di memorizzazione dei file (tipicamente 128MB o più). I file sono suddivisi in blocchi distribuiti sui DataNode.

YARN (Yet Another Resource Negotiator)

È il sistema di gestione delle risorse del cluster.
Classi/componenti principali:

- **ResourceManager :**

Gestisce le risorse globali del cluster e assegna i container.

- **NodeManager :**

Gestisce le risorse su ogni nodo (CPU, RAM) e monitora i container.

- **ApplicationMaster :**

Gestisce il ciclo di vita di una singola applicazione (es. job MapReduce), negozia le risorse e coordina i task.

- **Container :**

Unità di esecuzione (CPU + RAM) in cui girano i task.

MapReduce (modello di elaborazione)

È il framework per processare dati distribuiti.

Classi principali:

- **Mapper<KEYIN, VALUEIN, KEYOUT, VALUEOUT> :**

Trasforma i dati in input in coppie chiave-valore intermedie.

- **Reducer<KEYIN, VALUEIN, KEYOUT, VALUEOUT> :**

Aggrega i dati intermedi per produrre il risultato finale.

- **Job :**

Rappresenta una configurazione completa di esecuzione (input, output, mapper, reducer).

- **InputFormat :**

Definisce come leggere i dati (es. TextInputFormat legge riga per riga).

- **OutputFormat :**

Definisce come scrivere i risultati.

- **RecordReader :**

Converte i dati grezzi in coppie chiave-valore per il Mapper.

- **Partitioner :**

Decide quale Reducer riceve una certa chiave.

- **Writable :**

Interfaccia per la serializzazione dei dati (es. IntWritable, Text).

Ecco una lista dei principali metodi/operazioni di Hadoop (principalmente riferiti a MapReduce e HDFS), nel formato richiesto:
map() :

È il metodo principale della fase di mapping.

Prende in input una coppia chiave-valore e produce una o più coppie chiave-valore intermedie.

Serve per trasformare i dati grezzi in una forma elaborabile.

Parametri:

- **KEYIN → tipo della chiave in input**
- **VALUEIN → tipo del valore in input**
- **Context → oggetto che permette di scrivere l'output (key, value)**

reduce() :

Metodo della fase di riduzione.

Riceve una chiave e una lista di valori associati, aggregandoli per produrre il risultato finale.

È usato per operazioni come somma, conteggio o media.

Parametri:

- **KEYIN → chiave intermedia**
- **Iterable<VALUEIN> → lista dei valori associati alla chiave**
- **Context → oggetto per scrivere l'output finale**

setup() :

Metodo eseguito una sola volta all'inizio del task (mapper o reducer).

Serve per inizializzare variabili, connessioni o strutture dati necessarie all'elaborazione.

Parametri:

- **Context → contesto del task**

cleanup() :

Metodo eseguito una sola volta alla fine del task.

Serve per liberare risorse o fare operazioni finali (es. scrivere risultati accumulati).

Parametri:

- **Context → contesto del task**

run() :

Metodo che gestisce l'intero ciclo di vita del mapper o reducer (setup → map/reduce → cleanup).

Può essere sovrascritto per controllare il flusso completo.

Parametri:

- **Context → contesto di esecuzione**

write() (Context.write) :

Metodo usato per emettere coppie chiave-valore come output del mapper o reducer.

È il modo con cui i dati vengono passati alla fase successiva o salvati.

Parametri:

- **KEYOUT → chiave di output**
- **VALUEOUT → valore di output**

addInputPath() :

Metodo della classe FileInputFormat che definisce il percorso dei dati di input su HDFS.

Parametri:

- **Job → oggetto job Hadoop**
- **Path → percorso del file o directory di input**

setOutputPath() :

Metodo della classe FileOutputFormat che definisce dove salvare i risultati su HDFS.

Parametri:

- **Job → oggetto job Hadoop**
- **Path → percorso di output**

setMapperClass() :

Specifica quale classe Mapper deve essere utilizzata nel job.

Parametri:

- **Job → oggetto job**
- **Class → classe che estende Mapper**

setReducerClass() :

Specifica quale classe Reducer deve essere utilizzata nel job.

Parametri:

- **Job → oggetto job**
- **Class → classe che estende Reducer**

setOutputKeyClass() :

Definisce il tipo della chiave in output finale.

Parametri:

- **Class → tipo della chiave (es. Text, IntWritable)**

setOutputValueClass() :

Definisce il tipo del valore in output finale.

Parametri:

- **Class → tipo del valore (es. IntWritable, DoubleWritable)**

waitForCompletion() :

Avvia il job Hadoop e attende il completamento dell'esecuzione. Restituisce true se il job termina correttamente.

Parametri:

- **boolean verbose → se true mostra i log dettagliati**

waitForCompletion() :

Avvia il job Hadoop e attende il completamento dell'esecuzione. Restituisce true se il job termina correttamente.

Parametri:

- **boolean verbose → se true mostra i log dettagliati**

Pipeline base MapReduce (WordCount)

- Input salvato su HDFS (file diviso in blocchi)
- InputFormat legge i dati → RecordReader crea coppie (offset, riga)
- Mapper riceve (chiave, valore) → produce (parola, 1)
- Shuffle & Sort → raggruppa tutte le stesse parole
- Partitioner distribuisce le chiavi ai Reducer
- Reducer somma i valori → (parola, conteggio)
- OutputFormat scrive il risultato su HDFS

Pipeline con più step (job concatenati)

- **Job 1:**
  - Input HDFS → Mapper filtra e normalizza dati
  - Reducer aggrega parzialmente
  - Output intermedio su HDFS

- **Job 2:**
  - Input = output del Job 1
  - Mapper rielabora i dati (es. calcolo statistiche)
  - Reducer finale produce risultato globale
  - Output finale su HDFS

Pipeline con YARN (gestione completa)

- L’utente invia il Job al ResourceManager
- ResourceManager avvia un ApplicationMaster
- ApplicationMaster richiede container ai NodeManager
- I Mapper vengono eseguiti nei container
- Shuffle & Sort tra nodi
- I Reducer vengono eseguiti
- Risultati salvati su HDFS
- ApplicationMaster termina e rilascia le risorse

Buon Davante a tutti